

The Agentic SDLC Reference Architecture

Two layers are infrastructure. Three are where your team compounds.

AGENT PRIMITIVES Skills, Personas, Plugins – the prose-as-code your team writes.	COMPOUND
CONTEXT What the agent sees: codebase, docs, runtime signals, evals.	COMPOUND
GOVERNANCE & DISTRIBUTION How agents and primitives are versioned, reviewed, shipped, pinned.	COMPOUND
AGENT HARNESS Copilot CLI / Claude Code / Cursor / Codex. Commoditizing fast.	INFRASTRUCTURE
PLATFORM Models, inference, identity, telemetry – commoditizing infrastructure.	INFRASTRUCTURE

ONE AGENT SKILL PER SDLC PHASE

Each Skill: prose-as-code, versioned independently, pinned via lockfile.

IDEATE	PLAN	BUILD	TEST	REVIEW	DEPLOY	OPERATE
Discovery Skill v0.3.0 @your-org	Spec Skill v0.5.1 @your-org	Scaffold Skill v1.0.0 @your-org	TDD Skill v0.7.2 @your-org	Code-Review Skill v1.2.0 @your-org	Release Skill v0.4.1 @your-org	Incident Skill v0.2.0 @your-org

Same packaging discipline as Plugins below. The recursion is the point: any primitive can compose with any other.

DEPENDENCY GRAPH · EXAMPLE

```
graph TD
    A["code-review v1.2.0"] --> B["compliance-scan v0.4.1"]
    B --> C["policy-pack v0.2.0"]
```

Composable agent behavior. Pinned via lockfile.

THREE QUESTIONS FOR MONDAY

The ones I wish I'd been asked the first time I shipped an agent into production.

- 01Which lifecycle phase do you start with?
- 02Where do you invest in the supply chain before agents arrive?
- 03Build, buy, or compose – what's your posture per context domain?

THE THESIS

The model isn't the bottleneck. The harness isn't either. The engineering layer above both is where compounding lives – and most teams don't have one.

— from The Agentic SDLC Handbook, May 2026 edition